Ejercicio 1:

```c
#include <math.h>
#include <stdio.h>
void main(void) {
    signed char a, b, c, d, s1, s2;
    a = 127;
    b = 127;
    c = a + b;
    d = a * b;

    s1 = (-8) >> 2;
    s2 = (-1) >> 5;

    printf("a = %d \n", a );
    printf("b = %d \n", b );
    printf("c = %d \n", c );
    printf("d = %d \n", d );
    printf("s1 = %d \n", s1 );
    printf("s2 = %d \n", s2 ); }
```

```
garcia-14077@garcia-14077-desktop:~/Escritorio/Facultad/Quinto/CyS/TP2$ ./Ejercicio1
a = 127
b = 127
c = -2
d = 1
s1 = -2
s2 = -1
```

1. Verifique el valor de las variables c y d. ¿Son estos valores correctos?

No, los valores no son correctos.

- Para c (Suma): El valor esperado es 127 + 127 = 254. Sin embargo, un signed char de 8 bits tiene un rango de -128 a 127. Al intentar almacenar 254, ocurre un overflow (desbordamiento). El resultado en la terminal es de -2, debido al efecto de "caída en escalera" del complemento a 2
- Para d (Multiplicación): El valor esperado es 127*127 = 16129. Esto supera por mucho el límite de 127 de un signed char. El resultado es un valor erróneo (1) debido a que solo se conservan los 8 bits menos significativos del resultado real.

2. De no ser así, ¿qué soluciones propone?

Para evitar estos errores, se deben aplicar las técnicas que hemos visto en la teoría:

- Uso de un acumulador más largo: Declarar las variables de resultado (c y d) como tipos de mayor capacidad, por ejemplo int16_t (2 bytes) o int (4 bytes).
- Saturación: Implementar una función que, si el resultado supera 127, devuelva 127 en lugar de permitir el rollover.
- Casting: Realizar la operación convirtiendo los operandos a un tipo mayor antes de operar: c = (int)a + b;.

3. Verifique el valor de las variables s1 y s2. ¿Son estos valores correctos?

Sí, los valores obtenidos (s1 = -2 y s2 = -1) son correctos.

Al aplicar el operador de desplazamiento a la derecha (>>) sobre variables declaradas con signo (signed char), el compilador de C ejecuta un desplazamiento aritmético.

A diferencia del desplazamiento lógico, en el desplazamiento aritmético se preserva el bit de signo. Esto significa que, al desplazar los bits hacia la derecha, los espacios vacíos generados a la izquierda se rellenan con unos (1) por tratarse de números negativos.

- Para s1 = (-8) >> 2: Desplazar 2 posiciones a la derecha equivale a dividir por 2^2 (es decir, 4). Por lo tanto, -8 / 4 = -2.
- Para s2 = (-1) >> 5: El número -1 en complemento a 2 está formado por puros unos (11111111). Al desplazar aritméticamente hacia la derecha, se siguen introduciendo unos por la izquierda para preservar el signo, por lo que la palabra binaria no cambia y el resultado sigue siendo -1.

Ejercicio 2

Ejercicio_1_TP2.c Ejercicio_2_TP2.c

home > garcia-14077 > Escritorio > Facultad > Quinto > CyS > TP2 > Ejercicio_2_TP2.c > fp2fx(float, int)

```c
#include <stdio.h>
#include <stdint.h>

// 1. Funcion para pasar de punto fijo a punto flotante
float fx2fp(int32_t X, int n) {
    return (float)X / (1 << n);
}

// 2. Funcion para pasar de punto flotante a punto fijo
int32_t fp2fx(float x, int n) {
    return (int32_t)(x * (1 << n));
}

int main() {
    float valor_original = 2.4515;

    // 3. Verificacion para Q23.8 (n = 8 bits fraccionarios)
    float b_Q8 = fx2fp( fp2fx(valor_original, 8), 8 );

    // 3. Verificacion para Q21.10 (n = 10 bits fraccionarios)
    float b_Q10 = fx2fp( fp2fx(valor_original, 10), 10 );

    printf("Original: %f \n", valor_original);
    printf("b recuperado en Q23.8: %f \n", b_Q8);
    printf("b recuperado en Q21.10: %f \n", b_Q10);

    return 0;
}
```

```
garcia-14077@garcia-14077-desktop:~/Escritorio/Facultad/Quinto/CyS/TP2$ ./Ejercicio2
Original: 2.451500
b recuperado en Q23.8: 2.449219
b recuperado en Q21.10: 2.451172
```

Al comparar la variable 'b' con el valor original (2.4515), obtenemos los siguientes valores recuperados:

- En formato Q23.8: b = 2.449219
- En formato Q21.10: b = 2.451172
-

Se observa que ninguno de los dos formatos logra recuperar el número original con total exactitud debido a la pérdida de información que ocurre al truncar los decimales al pasar a número entero. Sin embargo, el valor recuperado con Q21.10 es mucho más cercano al original.

Esto sucede porque la precisión en punto fijo depende de la cantidad de bits asignados a la parte fraccionaria ("n").

- En el formato Q23.8, al tener 8 bits para la fracción, la precisión o resolución mínima es de 1/(2^8) = 0.00390625.
- En el formato Q21.10, tenemos 10 bits para la fracción, lo que mejora significativamente la precisión a 1/(2^10) = 0.0009765625.

Al tener una mayor precisión en Q21.10, el error de cuantización (o truncamiento) se reduce, permitiendo representar la parte decimal (0.4515) con mucha más fidelidad.

Ejercicio 3

Ejericio_1_TP2.c | Ejercicio_2_TP2.c | Ejercicio_3_TP2.c

home > garcia-14077 > Escritorio > Facultad > Quinto > CyS > TP2 > Ejercicio_3_TP2.c > main()

```c
#include <stdio.h>
#include <stdint.h>

// Se define la cantidad de bits fraccionarios para Q21.10
#define n 10

// 1. Redondeo por truncación , redondeo a menos infinito
int32_t truncation(int64_t X) {
    int32_t a;
    a = (int32_t)(X >> n); // Desplazamos a la derecha para eliminar los bits extra
    return a;
}

// 2. Redondeo al valor más cercano, se suma la mitad de la precisión y luego se trunca
int32_t rounding(int64_t X) {
    int64_t a;
    a = X + (1 << (n - 1)); // Se suma 2^(n-1)
    return truncation(a); // Reutilizamos la funcion de truncamiento
}

int main() {
    // Valores de prueba
    float val1 = 2.4515f;
    float val2 = 3.1234f;

    // Convertimos a Q21.10 (multiplicando por 2^10)
    int32_t q_val1 = (int32_t)(val1 * (1 << n));
    int32_t q_val2 = (int32_t)(val2 * (1 << n));

    // Multiplicación en punto fijo (requiere 64 bits para no desbordar)
    int64_t mult_result = (int64_t)q_val1 * (int64_t)q_val2;

    // Aplicamos los esquemas de redondeo
    int32_t res_trunc = truncation(mult_result);
    int32_t res_round = rounding(mult_result);

    // Volvemos a flotante para comparar
    float f_trunc = (float)res_trunc / (1 << n);
    float f_round = (float)res_round / (1 << n);

```

```c

    // 3. Resultado exacto usando double
    double real_double = (double)val1 * (double)val2;

    printf("Resultado en double: %f \n", real_double);
    printf("Resultado con truncacion: %f \n", f_trunc);
    printf("Resultado con redondeo: %f \n", f_round);

    return 0;
}
```

```
garcia-14077@garcia-14077-desktop:~/Escritorio/Facultad/Quinto/CyS/TP2$ ./Ejercicio3
Resultado en double: 7.657015
Resultado con truncacion: 7.654297
Resultado con redondeo: 7.655273
```

Al comparar el resultado de la multiplicación en formato Q21.10 usando ambos esquemas frente al cálculo nativo en double, se observa lo siguiente:

- Truncamiento: El valor obtenido siempre presenta un error mayor. Esto se debe a que la truncación elimina los 10 bits menos significativos generados por la multiplicación (desplazamiento a la derecha), lo que matemáticamente equivale a un redondeo hacia menos infinito. Toda la información decimal residual simplemente se pierde.
- Redondeo al valor más cercano: El valor obtenido es mucho más fiel al resultado arrojado por el formato double. Al sumar la mitad de la precisión (1 << (n-1)) antes de realizar el desplazamiento, el algoritmo evalúa si los bits que se van a descartar superan la mitad del paso de cuantización. Si lo hacen, el valor se redondea hacia arriba, minimizando el error de cuantización en comparación con la simple truncación.
- Double: Es el valor de referencia ideal, ya que al usar un formato de punto flotante de 64 bits (IEEE 754), el error de representación para esta escala de números es prácticamente nulo.

Ejercicio 4

Ejercicio_1_TP2.c | Ejercicio_2_TP2.c | Ejercicio_3_TP2.c | Ejercicio_4_TP2.c

home > garcia-14077 > Escritorio > Facultad > Quinto > CyS > TP2 > Ejercicio_4_TP2.c > main()

```c
#include <stdio.h>
#include <stdint.h>
#include <limits.h>

// 1. Funcion de suma con saturacion
int32_t saturation(int32_t a, int32_t b) {
    // Primero operamos en un acumulador mas largo (64 bits) para que no desborde al sumar
    int64_t suma = (int64_t)a + (int64_t)b;

    // Verificamos los limites y saturamos si es necesario
    if (suma > INT_MAX) {
        return INT_MAX;
    } else if (suma < INT_MIN) {
        return INT_MIN;
    } else {
        return (int32_t)suma;
    }
}

int main() {
    // 2. Serie de numeros para verificar el funcionamiento
    // Caso A: Suma normal (sin overflow)
    int32_t normal_1 = 150000;
    int32_t normal_2 = 250000;
    // Caso B: Overflow positivo (2000 millones + 1000 millones > 2147483647)
    int32_t over_pos_1 = 2000000000;
    int32_t over_pos_2 = 1000000000;
    // Caso C: Overflow negativo (-2000 millones - 1000 millones < -2147483648)
    int32_t over_neg_1 = -2000000000;
    int32_t over_neg_2 = -1000000000;

    printf("--- Verificacion de Saturacion ---\n");
    printf("Suma normal: %d + %d = %d \n", normal_1, normal_2, saturation(normal_1, normal_2));
    printf("Suma con overflow positivo: %d + %d = %d (Saturado a INT_MAX)\n", over_pos_1, over_pos_2, saturation(over_pos_1, over_pos_2));
    printf("Suma con overflow negativo: %d + %d = %d (Saturado a INT_MIN)\n", over_neg_1, over_neg_2, saturation(over_neg_1, over_neg_2));

    return 0;
}
```

```
garcia-14077@garcia-14077-desktop:~/Escritorio/Facultad/Quinto/CyS/TP2$ ./Ejercicio4
--- Verificacion de Saturacion ---
Suma normal: 150000 + 250000 = 400000
Suma con overflow positivo: 2000000000 + 1000000000 = 2147483647 (Saturado a INT_MAX)
Suma con overflow negativo: -2000000000 + -1000000000 = -2147483648 (Saturado a INT_MIN)
```

La función saturation() castea primero los sumandos a 64 bits para realizar la operación de forma segura. Si el resultado (suma) supera el valor máximo que puede almacenar un entero de 32 bits con signo (INT_MAX), la función devuelve INT_MAX, "recortando" el valor. Lo mismo hace en el límite inferior con INT_MIN. Esto introduce una no linealidad en el sistema, pero evita la discontinuidad catastrófica del overflow clásico.

Ejercicio 5

Ejercicio_1_TP2.c | Ejercicio_2_TP2.c | Ejercicio_3_TP2.c | Ejercicio_4_TP2.c | Ejercicio_5_TP2.c

home > garcia-14077 > Escritorio > Facultad > Quinto > CyS > TP2 > Ejercicio_5_TP2.c > main()

```c
#include <stdio.h>
#include <stdint.h>

#define n 10

// Funcion de truncamiento
int32_t truncation(int64_t X) {
    return (int32_t)(X >> n);
}

int main() {
    // 1 y 4. Vectores originales
    float X_f[5] = {1.1, 2.2, 3.3, 4.4, 5.5};
    float Y_f[5] = {6.6, 7.7, 8.8, 9.9, 10.10};

    double X_d[5] = {1.1, 2.2, 3.3, 4.4, 5.5};
    double Y_d[5] = {6.6, 7.7, 8.8, 9.9, 10.10};

    // 2. Conversion a formato Q21.10
    int32_t X_q[5];
    int32_t Y_q[5];
    for(int i = 0; i < 5; i++) {
        X_q[i] = (int32_t)(X_f[i] * (1 << n));
        Y_q[i] = (int32_t)(Y_f[i] * (1 << n));
    }

    // Variables para la operacion MAC
    int32_t acum_32a = 0;
    int64_t acum_64 = 0;
    int32_t acum_32b = 0;
    double acum_db = 0.0;

    // 3. Implementacion de los enfoques MAC
    for(int i = 0; i < 5; i++) {
        // Enfoque 1: truncar antes de sumar
        acum_32a += (int32_t)(truncation((int64_t)X_q[i] * Y_q[i]));

        // Enfoque 2: acumular en 64 bits sin truncar
        acum_64 += (int64_t)X_q[i] * Y_q[i];

        // Operacion de referencia en double
        acum_db += X_d[i] * Y_d[i];
    }

    // Truncamiento final para el enfoque 2
    acum_32b = (int32_t)truncation(acum_64);

    // Impresion de resultados convertidos a flotante para visualizacion
    printf("acum_32a (Truncamiento previo): %f\n", (float)acum_32a / (1 << n));
    printf("acum_32b (Truncamiento final): %f\n", (float)acum_32b / (1 << n));
    printf("acum_db (Formato double): %f\n", acum_db);

    return 0;
}
```

```
garcia-14077@garcia-14077-desktop:~/Escritorio/Facultad/Quinto/CyS/TP2$ ./Ejercicio5
acum_32a (Truncamiento previo): 152.324219
acum_32b (Truncamiento final): 152.326172
acum_db (Formato double): 152.350000
```

Al comparar los resultados de la operación MAC (multiplicaciones y acumulación), se observa una diferencia en la precisión final según el momento en que se aplica el truncamiento:

- Enfoque 1 (acum_32a): La pérdida de precisión por el truncamiento ocurre dentro del bucle en cada una de las 5 multiplicaciones. Estos errores de redondeo hacia menos infinito se acumulan iteración tras iteración, provocando que el resultado final se aleje del valor real.

- Enfoque 2 (acum_32b): Se utiliza un acumulador más largo (64 bits) para mantener la resolución completa que resulta de multiplicar dos números de 32 bits. El descarte de bits fraccionarios se realiza una única vez al finalizar todas las sumas.

- Conclusión: El valor de acum_32b es más exacto que acum_32a y presenta una menor desviación respecto al valor de referencia ideal calculado en formato double (acum_db). Este es el motivo por el cual los Procesadores Digitales de Señales (DSP) ejecutan las operaciones MAC utilizando acumuladores extendidos (con bits de guarda).

Ejercicio 6

Para implementar un filtro digital, la operación básica que ejecuta el procesador es la operación MAC (Multiplicaciones y Acumulación). En un procesador DSP de 16 bits, las muestras de la señal y los coeficientes del filtro tienen una longitud de 16 bits.

Al multiplicar dos números de 16 bits (por ejemplo, muestra por coeficiente), se genera un producto de 32 bits. Por lo tanto, los valores que realmente se van a sumar y acumular en la ALU tienen un tamaño de N = 32 bits.

Para evitar el overflow al sumar "s" cantidad de números de "N" bits, el acumulador debe tener un tamaño total de: N + log2(s) bits.

A partir de esta ecuación, podemos calcular la cantidad máxima de sumas (s):

- Tamaño total del acumulador = 40 bits
- Tamaño de cada sumando (producto) = 32 bits
- Bits de guarda = 40 - 32 = 8 bits

Planteamos la ecuación: 40 = 32 + log2(s); 8 = log2(s); s = 2^8; s = 256

Respuesta: Se pueden realizar 256 sumas consecutivas garantizando que no se producirá un overflow. Los 8 bits adicionales del acumulador (por encima de los 32 bits del producto) funcionan de manera dedicada como "bits de guarda", permitiendo acumular hasta 256 términos de un filtro digital sin riesgo de desbordamiento de memoria.

(Nota de justificación: Si el ejercicio se refiriera a sumar directamente los datos de 16 bits de entrada sin multiplicarlos previamente, la ecuación sería 40 = 16 + log2(s), lo que permitiría 16.777.216 sumas. Sin embargo, al tratarse de la implementación de un filtro digital, se asume la operación MAC donde se acumulan productos de 32 bits).